

● AI ENGINEERING JOURNEY / FIELD NOTEBOOK

RAG Engineering Notebook

Retrieval-Augmented Generation — architecture, pipeline stages, chunking strategies, এবং RAG বনাম Fine-Tuning নিয়ে একটি সম্পূর্ণ engineering report, বাংলা ব্যাখ্যাসহ।

DOCUMENT

RAG (Retrieval-Augmented
Generation) Report

PREPARED

14 August 2026

SECTIONS

19 — Concept থেকে Reflection
পর্যন্ত

LANGUAGE

বাংলা ব্যাখ্যা + English technical
terms

সূচিপত্র

Table of Contents

- 01 RAG আসলে কী?
- 02 কেন RAG দরকার?
- 03 RAG-এর মূল Architecture
- 04 RAG-এর দুইটি প্রধান Phase
- 05 RAG Process — সম্পূর্ণ পদক্ষেপ
- 06 Step 1 — Data Ingestion
- 07 Step 2 — PDF Parsing
- 08 Step 3 — Chunking
- 09 কেন Chunking গুরুত্বপূর্ণ?
- 10 Chunking Strategies
- 11 Step 4 — Embedding
- 12 Embedding কেন প্রয়োজন?
- 13 Vector Storage
- 14 Retrieval
- 15 Top-K Retrieval
- 16 Step 5 — Augmentation
- 17 Step 6 — Generation
- 18 RAG বনাম Fine-Tuning
- 19 আমি RAG সম্পর্কে যা বুঝেছি

01

CORE CONCEPT

RAG আসলে কী?

RAG হলো এমন একটি architecture যেখানে একটি Large Language Model (LLM) উত্তর তৈরি করার আগে external knowledge source থেকে প্রয়োজনীয় তথ্য retrieve করে এবং সেই তথ্যকে context হিসেবে ব্যবহার করে উত্তর তৈরি করে।

Example

ধরা যাক একটি কোম্পানির knowledge base-এ আছে:

- employee-handbook.pdf
- company-policy.pdf
- product-documentation.pdf
- pricing.pdf

User প্রশ্ন করল:

"How many annual leave days do employees receive?"

RAG system:

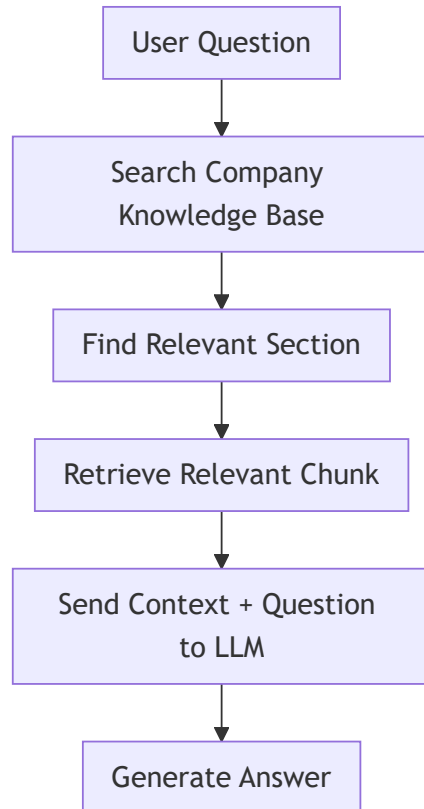


FIG 1.1 – একটি সাধারণ RAG request flow: প্রশ্ন থেকে উত্তর পর্যন্ত।

সম্ভাব্য answer:

"According to the employee handbook, employees receive 20 days of annual leave per year."

এখানে LLM-এর মূল কাজ হলো retrieved information ব্যবহার করে উত্তর তৈরি করা।

কেন RAG দরকার?

একটি সাধারণ LLM অনেক ধরনের knowledge ধারণ করতে পারে, কিন্তু application-specific knowledge-এর ক্ষেত্রে সীমাবদ্ধতা থাকে।

2.1 Private / Proprietary Data

একটি company's private information সাধারণত public LLM knowledge-এর অংশ নয়। যেমন:

- Internal HR Policy
- Internal API Documentation
- Customer Support Knowledge Base
- Company Pricing
- Internal SOP
- Private Product Documentation

RAG ব্যবহার করে এই information external knowledge base-এ রাখা যায় এবং প্রয়োজন অনুযায়ী retrieve করা যায়।

2.2 Frequently Changing Information

কিছু information নিয়মিত পরিবর্তিত হয়। যেমন:

- Product Price
- Inventory
- Company Policy
- Product Documentation
- FAQ
- Service Availability

2.3 Grounded Answers

LLM কখনো এমন information generate করতে পারে যার কোনো বাস্তব source নেই। RAG-এর মাধ্যমে relevant context LLM-কে দেওয়া যায়।

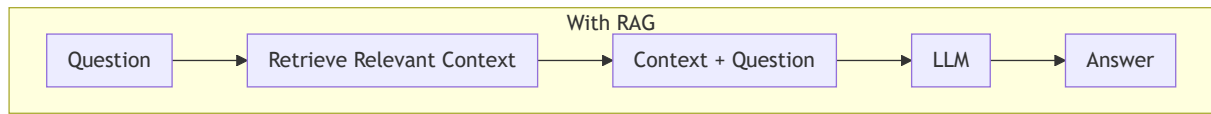


FIG 2.1 – RAG থাকলে উত্তর দেওয়ার আগে relevant context যুক্ত হয়।

RAG hallucination সম্পূর্ণভাবে বন্ধ করে না। যদি retrieval ভুল হয়, তাহলে LLM ভুল context-এর ওপর ভিত্তি করে ভুল উত্তর দিতে পারে।

2.4 Source-based Answers

একটি ভালো RAG application answer-এর সাথে source information-ও দিতে পারে। Example:

Answer:
Employees receive 20 days of annual leave per year.

Source:
employee-handbook.pdf
Page: 12

এটি enterprise application-এর জন্য গুরুত্বপূর্ণ, কারণ user answer-এর source verify করতে পারে।

RAG-এর মূল Architecture

RAG architecture-কে মূলত দুইটি flow হিসেবে বোঝা যায়।

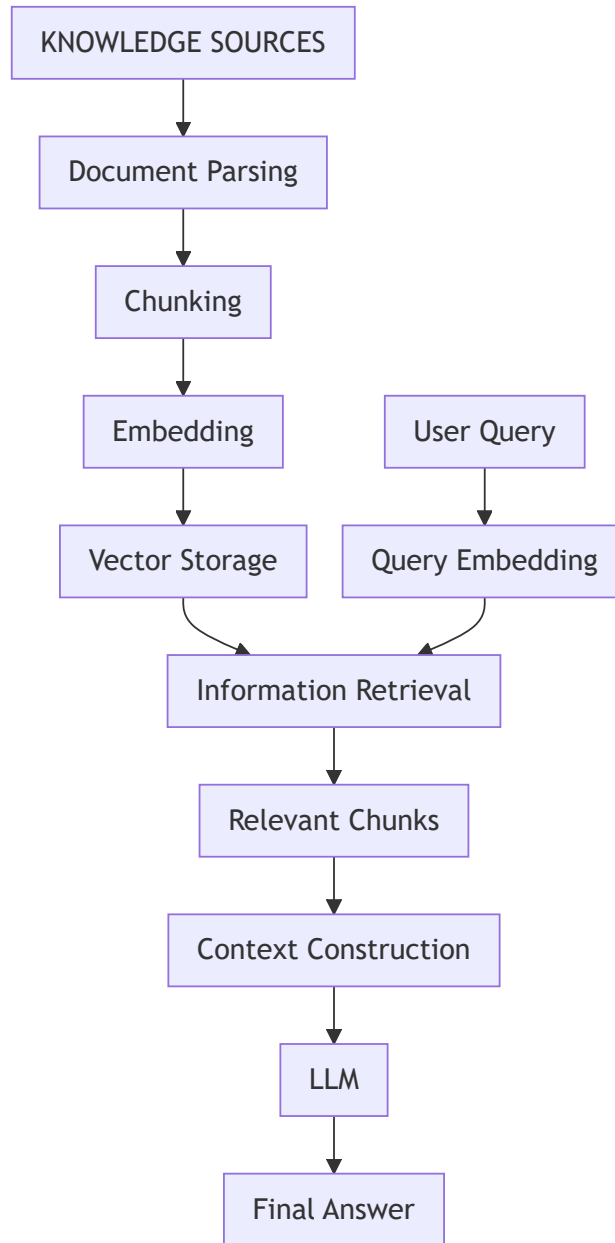


FIG 3.1 – Indexing flow (উপরে) এবং Query flow (নিচে) একসাথে মিলিত হয়ে Final Answer তৈরি করে।

04

TWO PHASES

RAG-এর দুইটি প্রধান Phase

Phase A — Indexing / Ingestion

Indexing phase-এর উদ্দেশ্য হলো raw knowledge source-কে এমনভাবে প্রস্তুত করা যাতে পরবর্তীতে দ্রুত এবং relevant information retrieve করা যায়।

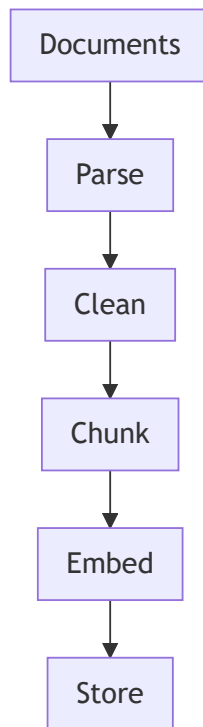


FIG 4.1 – Indexing-এর সাধারণ flow।

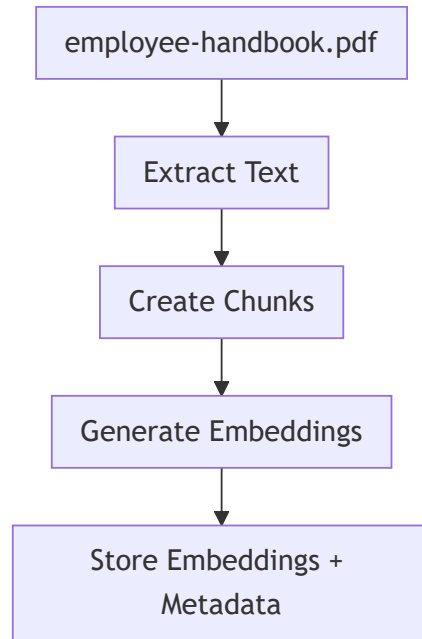


FIG 4.2 – বাস্তব উদাহরণ: employee-handbook.pdf ইনডেক্সিং।

এই phase সাধারণত document প্রথমবার system-এ যোগ করার সময় অথবা document update হলে চলে।

Phase B — Retrieval + Generation

User যখন প্রশ্ন করে, তখন query phase শুরু হয়।

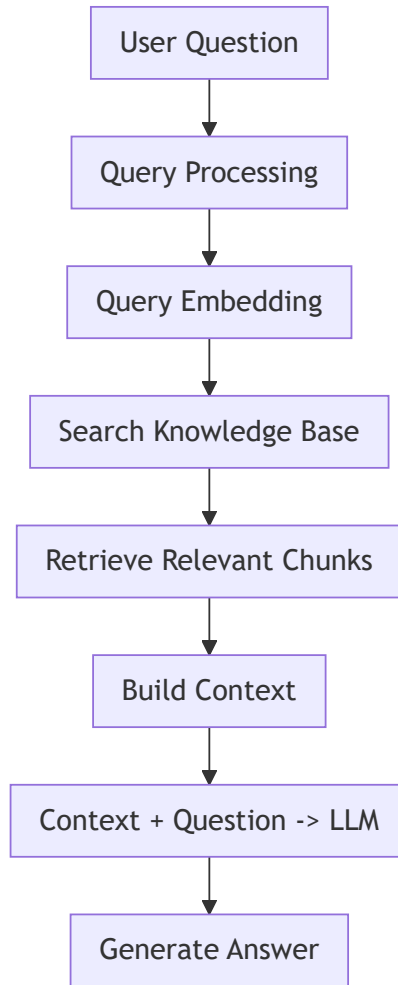


FIG 4.3 – Query phase-এর flow।

User-এর প্রশ্নের উত্তর দেওয়ার জন্য knowledge base-এর কোন অংশ প্রয়োজন তা খুঁজে বের করা — এখানে মূল কাজ।

05

END-TO-END

RAG Process

RAG-এর complete process কয়েকটি ধাপে বোঝা যায়। এখানে Retrieval-কে আলাদা করে বোঝা গুরুত্বপূর্ণ, কারণ এটি RAG-এর core component।

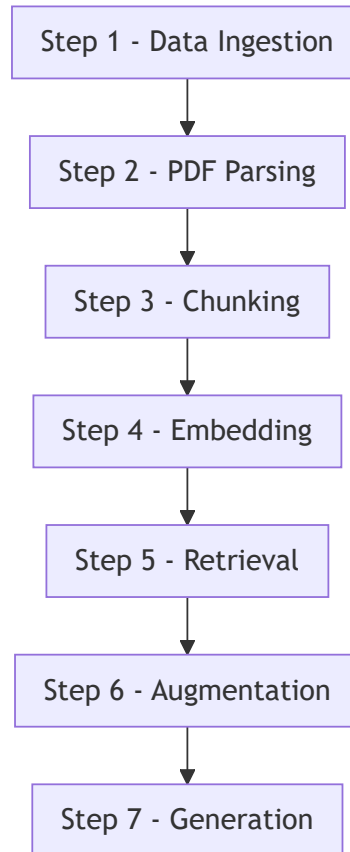


FIG 5.1 – সাত ধাপের সম্পূর্ণ RAG pipeline; পরবর্তী সেকশনগুলোতে প্রতিটি ধাপ আলাদাভাবে ব্যাখ্যা করা হয়েছে।

06

STEP 1

Data Ingestion

Data ingestion বলতে বিভিন্ন knowledge source-কে RAG system-এর মধ্যে আনা এবং processing-এর জন্য প্রস্তুত করাকে বোঝায়।

Possible sources:

- PDF
- DOCX
- TXT
- Markdown
- HTML
- Web Pages
- Database
- API
- Google Drive
- Notion
- GitHub
- Images

07

STEP 2

PDF Parsing

PDF parsing হলো PDF file থেকে usable information বের করে আনা। একটি PDF-এর মধ্যে থাকতে পারে:

- Text
- Tables
- Images
- Headers
- Footers
- Page Numbers
- Formatting
- Layout Information

Basic parsing flow:

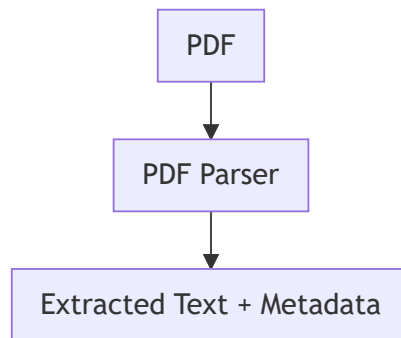


FIG 7.1 – PDF parsing-এর basic flow।

Example output:

```
{
  "text": "Employees are entitled to 20 days of annual leave...",
  "page": 12,
  "document": "employee-handbook.pdf"
}
```

Parsing-এর output পরবর্তী ধাপের input হিসেবে ব্যবহার করা হয়।

08

STEP 3

Chunking

একটি সম্পূর্ণ document সাধারণত retrieval-এর জন্য একটিমাত্র বড় block হিসেবে ব্যবহার করা হয় না। Document-কে ছোট ছোট অংশে ভাগ করা হয়। এই অংশগুলোকে **chunk** বলা হয়।

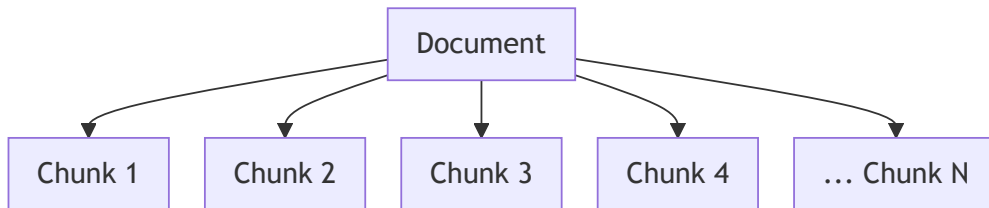


FIG 8.1 – একটি document একাধিক chunk-এ ভাগ হয়।

CHUNKING-এর মূল উদ্দেশ্য

- Information-কে manageable করা
- Relevant অংশ retrieve করা
- Context quality বজায় রাখা
- Unnecessary token usage কমানো

কেন Chunking গুরুত্বপূর্ণ?

ধরা যাক একটি PDF-এ 100 pages আছে। User প্রশ্ন করল:

"How many annual leave days are available?"

এখানে পুরো 100 pages LLM-এর কাছে পাঠানোর প্রয়োজন নেই। Relevant section যথেষ্ট:

Employees receive 20 days of annual leave per year.

তাই:

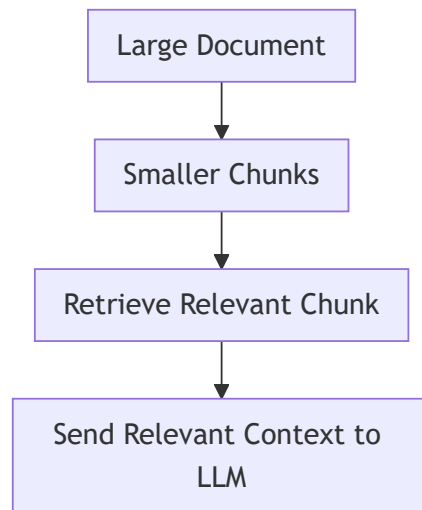


FIG 9.1 – বড় document থেকে শুধু relevant chunk পাঠানো হয়।

খারাপ chunking হলে retrieval ভালো হলেও relevant context পাওয়া নাও যেতে পারে।

Chunking Strategies

চারটি basic chunking strategy বোঝা গুরুত্বপূর্ণ:

1. Fixed-size Chunking
2. Overlapping Chunking
3. Semantic Chunking
4. Hierarchical Chunking

10.1 Fixed-size Chunking

Fixed-size chunking-এ নির্দিষ্ট size অনুযায়ী document ভাগ করা হয়। Example: Chunk Size = 500 tokens

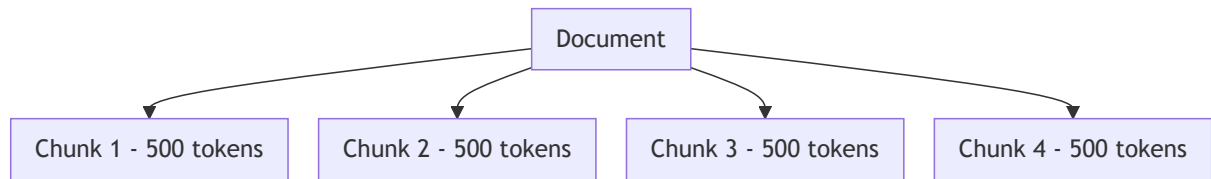


FIG 10.1 – Fixed-size chunking।

সুবিধা

- Implement করা সহজ
- Predictable
- Size control করা সহজ

সমস্যা

Sentence বা concept মাঝখানে split হয়ে যেতে পারে।

Chunk 1:
Employees are entitled to 20 days of annual

Chunk 2:
leave per year...

এখানে একটি complete idea দুইটি chunk-এ ভাগ হয়ে গেছে।

10.2 Overlapping Chunks

Chunk boundary problem কমানোর জন্য overlapping chunks ব্যবহার করা যায়।

```
Chunk 1 = Token 1-500
Chunk 2 = Token 400-900
Chunk 3 = Token 800-1300
```

এখানে neighbouring chunks-এর মধ্যে কিছু অংশ overlap করে:

```
Chunk 1
[-----]

      Chunk 2
      [-----]

            Chunk 3
            [-----]
```

সুবিধা

- Boundary information preserve করতে সাহায্য করে
- Sentence/context split হওয়ার সম্ভাবনা কমে

সমস্যা

- বেশি storage প্রয়োজন
- বেশি embeddings তৈরি হয়
- Processing cost বাড়তে পারে
- Duplicate information retrieval হতে পারে

10.3 Semantic Chunking

Semantic chunking token count-এর পরিবর্তে meaning বা logical structure অনুযায়ী document ভাগ করে।

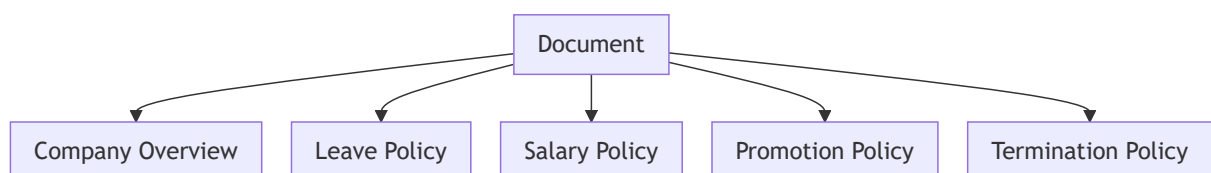


FIG 10.2 – Meaning অনুযায়ী section-ভিত্তিক chunking!

সুবিধা

- Meaning preserve করা সহজ
- Context বেশি coherent হয়
- Structured documents-এর জন্য useful

সমস্যা

- Implementation বেশি complex
- Semantic boundary নির্ধারণের জন্য অতিরিক্ত logic প্রয়োজন

10.4 Hierarchical Chunking

Hierarchical chunking document-এর structure বা hierarchy preserve করে।

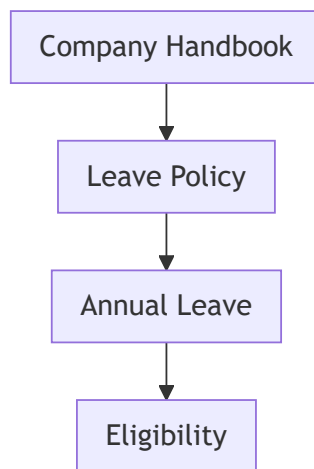


FIG 10.3 – একটি simple hierarchy।

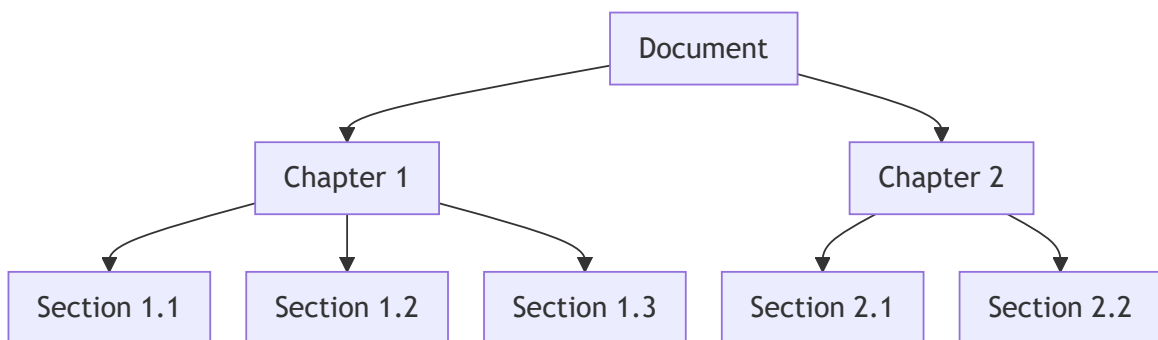


FIG 10.4 – আরও বড়, multi-level structure।

এই hierarchy metadata বা context হিসেবে preserve করা যেতে পারে। এটি complex বা highly structured documents-এর ক্ষেত্রে useful।

11

STEP 4

Embedding

Embedding হলো text-এর semantic meaning-কে একটি numerical vector representation-এ রূপান্তর করার প্রক্রিয়া।

Example:

```
"How many annual leave days do employees get?"
```

Conceptually এর embedding হতে পারে:

```
[0.021, -0.183, 0.742, 0.091, ...]
```

আরেকটি semantically similar sentence:

```
"Employees receive 20 vacation days per year."
```

এর vector-ও একই ধরনের semantic space-এ relatively কাছাকাছি হতে পারে। মূল উদ্দেশ্য হলো exact keyword match নয়, বরং **semantic similarity** ব্যবহার করে relevant information খুঁজে বের করা।

Embedding কেন প্রয়োজন?

ধরা যাক document-এ লেখা:

"Employees receive 20 vacation days per year."

কিন্তু user প্রশ্ন করল:

"How much time off can I take?"

এখানে exact word match নেই। তবুও দুইটির meaning কাছাকাছি:

- vacation
- annual leave
- time off
- holiday

Embedding এই semantic relationship capture করতে সাহায্য করে।

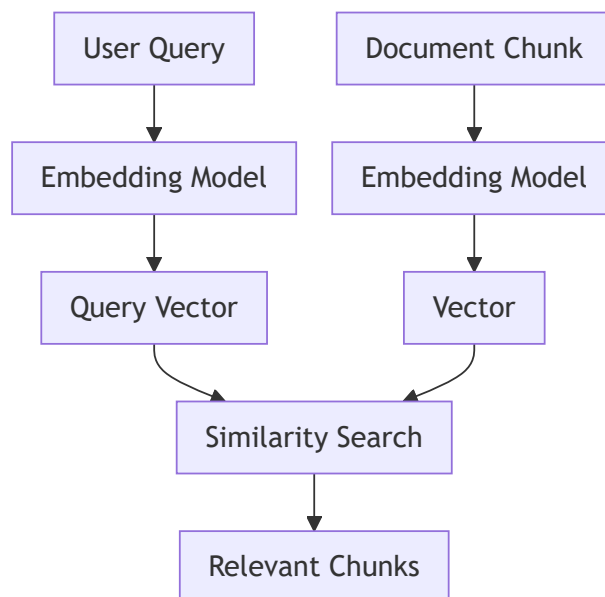


FIG 12.1 – Document embedding ও query embedding একই vector space-এ compare হয়।

13

STORAGE

Vector Storage

Embedding তৈরি করার পরে এগুলো searchable storage-এ রাখা হয়।

Common options:

- PostgreSQL + pgvector
- Qdrant
- Pinecone
- Weaviate
- Milvus
- Elasticsearch / OpenSearch
- FAISS

একটি record conceptually এমন হতে পারে:

```
{
  "id": "chunk_102",
  "embedding": [0.12, -0.43, 0.88],
  "text": "Employees receive 20 days...",
  "metadata": {
    "document": "employee-handbook.pdf",
    "page": 12,
    "department": "HR"
  }
}
```

এখানে তিনটি বিষয় গুরুত্বপূর্ণ: **Embedding, Text, Metadata** — embedding search-এর জন্য, original text context তৈরির জন্য এবং metadata filtering/source tracking-এর জন্য ব্যবহার করা যায়।

Retrieval হলো user-এর query অনুযায়ী knowledge base থেকে relevant information খুঁজে বের করার প্রক্রিয়া।

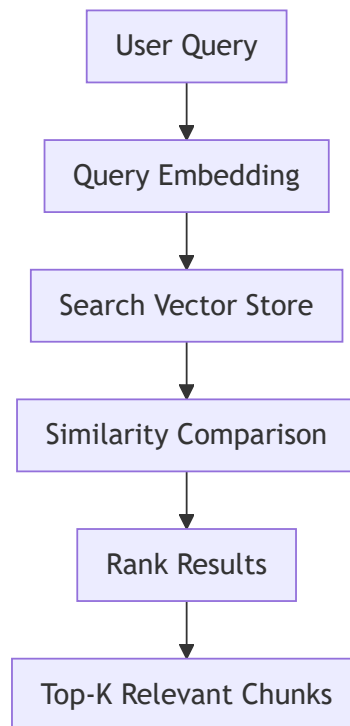


FIG 14.1 – Retrieval pipeline

Example ranking:

1. Chunk 102 → 0.94
2. Chunk 88 → 0.91
3. Chunk 301 → 0.86
4. Chunk 77 → 0.81
5. Chunk 410 → 0.79

এখানে similarity score যত বেশি, সাধারণভাবে chunk-টি query-এর সাথে তত বেশি relevant।

Top-K Retrieval

Top-K বলতে retrieval-এর পরে কতগুলো relevant result রাখা হবে তা বোঝায়। যদি $\text{top_k} = 5$, তাহলে system 5টি relevant chunk নির্বাচন করতে পারে।

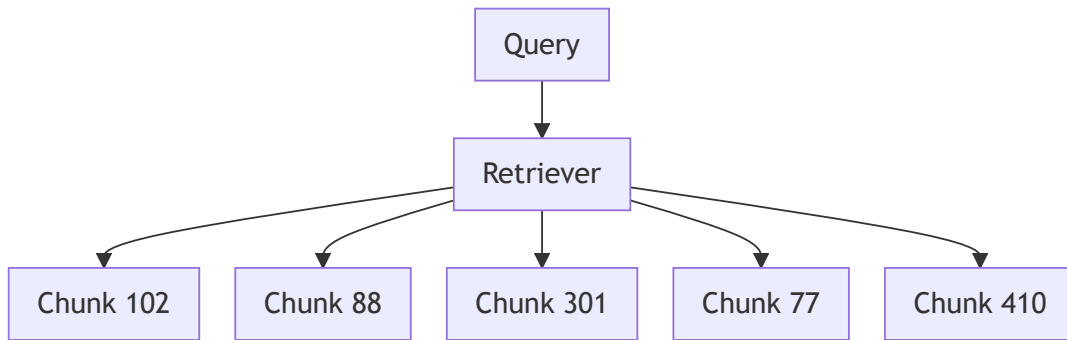


FIG 15.1 – $\text{top_k} = 5$ হলে Retriever পাঁচটি chunk রিটার্ন করে।

তবে top_k সবসময় একটি fixed universal value নয়। Dataset, chunk size, question type এবং context window অনুযায়ী এটি tune করতে হয়।

16

STEP 5

Augmentation

Retrieval-এর মাধ্যমে relevant information পাওয়ার পরে সেই information LLM-এর input-এর সাথে যুক্ত করা হয়।

System Instruction:

Answer using the provided context.

Context:

Employees are entitled to 20 days of annual leave per year.

Question:

How many annual leave days do employees get?

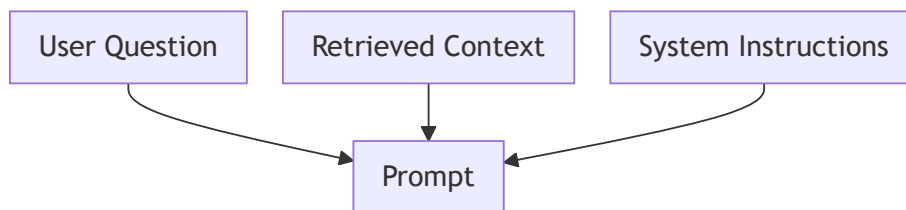


FIG 16.1 – তিনটি অংশ মিলে একটি Prompt তৈরি হয় – এটাই Augmentation stage।

Generation হলো LLM-এর মাধ্যমে final answer তৈরি করার ধাপ।

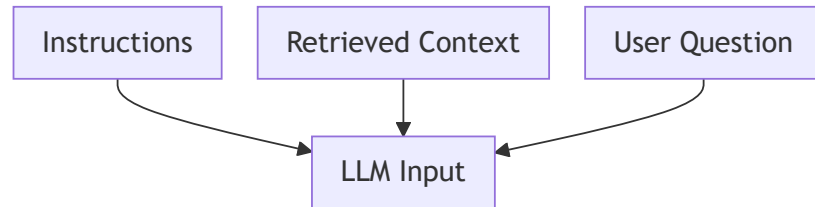


FIG 17.1 – LLM input-এর তিনটি অংশ।

Output:

"Employees are entitled to 20 days of annual leave per year."

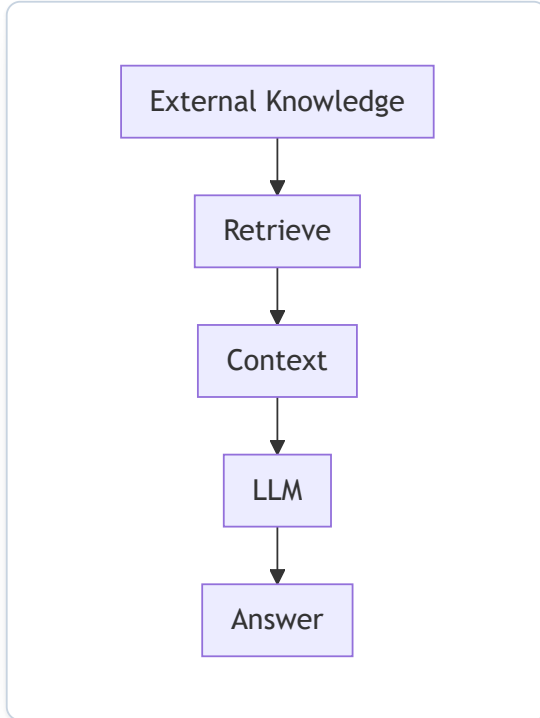
এখানে LLM-এর দায়িত্ব হলো retrieved information ব্যবহার করে coherent এবং useful natural-language response তৈরি করা।

RAG বনাম Fine-Tuning

RAG এবং Fine-Tuning একই কাজ করে না। দুটির উদ্দেশ্য আলাদা।

18.1 – RAG

RAG-এ knowledge model-এর বাইরে থাকে।

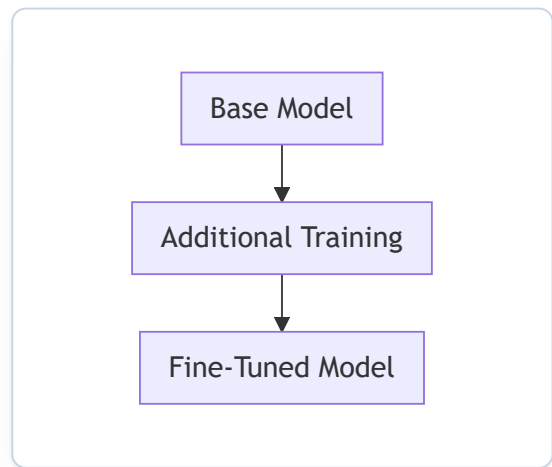


RAG সাধারণত USEFUL যখন

- Private documents নিয়ে কাজ করতে হবে
- Data frequently পরিবর্তিত হয়
- Company knowledge base তৈরি করতে হবে
- Source/citation প্রয়োজন
- Large document collection থেকে answer দিতে হবে

18.2 – FINE-TUNING

Fine-tuning-এর মাধ্যমে model-কে additional training data দিয়ে তার behavior বা task performance পরিবর্তন করা হয়।



FINE-TUNING USEFUL হতে পারে

- Specific response style
- Specific behavior
- Specialized task
- Domain-specific patterns

আমি RAG সম্পর্কে যা বুঝেছি

RAG শুধুমাত্র PDF upload করে LLM-কে প্রশ্ন করার ব্যবস্থা নয়। এটি একটি complete information retrieval এবং generation pipeline।

আমার বোঝাপড়া অনুযায়ী

1. Knowledge source সংগ্রহ করা হয়।
2. Document parse করে usable content তৈরি করা হয়।
3. বড় document ছোট meaningful chunk-এ ভাগ করা হয়।
4. প্রতিটি chunk-এর embedding তৈরি করা হয়।
5. Embedding, text এবং metadata searchable storage-এ রাখা হয়।
6. User-এর question process করা হয়।
7. Question-এর জন্য relevant information retrieve করা হয়।
8. Retrieved information context হিসেবে LLM-এর input-এর সাথে যুক্ত করা হয়।
9. LLM context এবং question ব্যবহার করে answer generate করে।
10. প্রয়োজন হলে source information-ও response-এর সাথে দেখানো যায়।